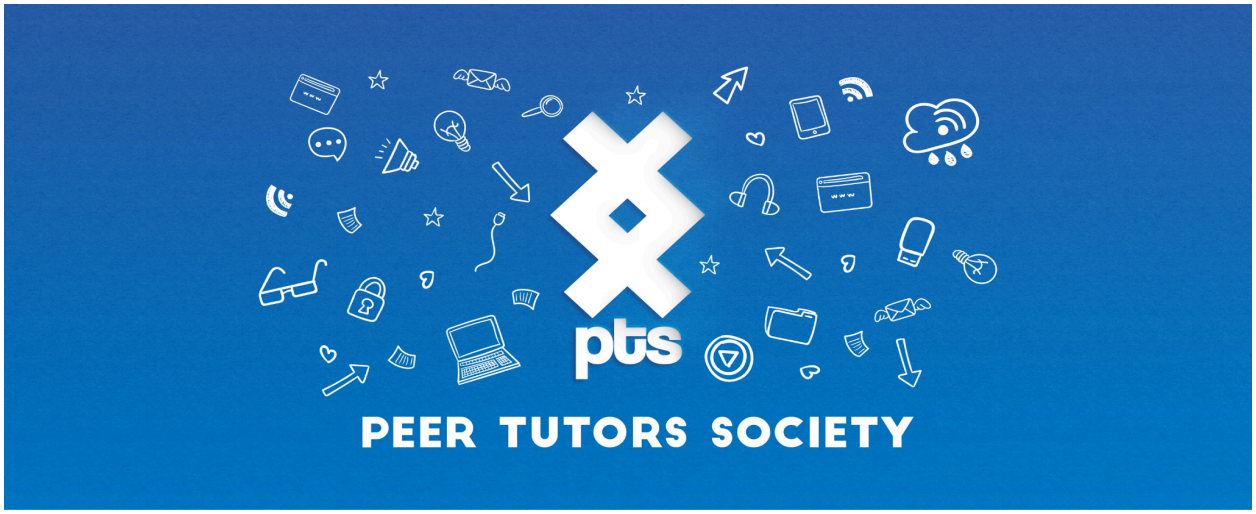
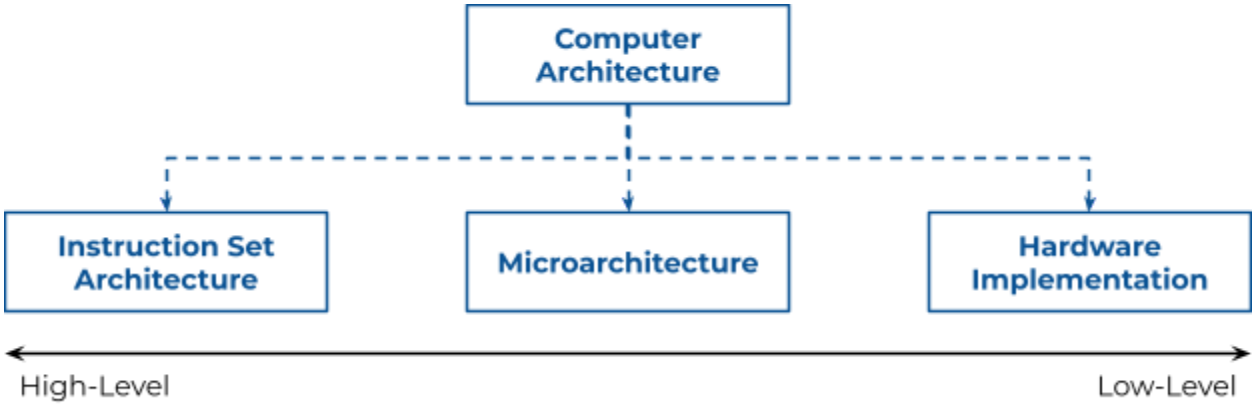


CSARCH2
Long Exam 1: Computer Organization and Von Neumann Architecture
Name of Contributor(s) :
Clive Jarel Ang

Note: This reviewer is intended for distribution under the name of PTS. Only the contributor or the organization may distribute it directly. For distribution by other organizations, please contact the Peer Tutors Society. For any concerns, reach out to the Peer Tutors Society's Academics Committee.

DLSU PTS Facebook
https://www.facebook.com/pts.dlsu/
DLSU PTS Website
https://dlsupts.vercel.app/

Introduction to Computer Organization



Computer Architecture

- Defines the conceptual structure and functional behavior of a computer system as seen by a programmer.
- Acts as the blueprint of a computing system, describing what the system should do rather than how it is implemented.

Instruction Set Architecture (ISA)

The part of computer architecture that is visible to the programmer. It **defines the instructions a processor can execute** and the way a programmer interacts with the system.

- The main components of the ISA include:
    - **Instruction Set**: The set of commands a processor understands, such as ADD, SUB, MUL, or LOAD.
        - **RISC (Reduced Instruction Set Computing)**: Simple and fixed-length instructions.
        - **CISC (Complex Instruction Set Computing)**: Complex and variable-length instructions.
    - **Addressing Modes**: How operands are accessed (immediate, direct, indirect, indexed).
    - **Registers**: Small, high-speed storage within CPU used for execution.
    - **Data Types**: Defines how numbers and characters are represented (integer, floating-point).

Computer Organization

Refers to the implementation details of the ISA. It determines **how the ISA is realized in hardware**, including the design of components like the processor, memory hierarchy, and control logic.

- The main components of computer organization include:
  - **Control Signals**: Timing and sequencing of operations.
  - **Datapath Design**: How data moves between ALU, registers, and memory.
  - **Memory Hierarchy**: Cache, main memory, and secondary storage design.
  - **I/O Organization**: Bus systems, device controllers, and interrupts.

Hardware Implementation

Refers to the **physical realization of the architecture**. It involves designing and fabricating the actual circuits, chips, and boards that make up the computer.

- The different levels of hardware implementation include:
  - **Digital Logic Level**: Logic gates (AND, OR, NOT) and flip-flops.
    - Example: Building a 1-bit full adder circuit.
  - **Microarchitecture Level**: Designing datapath, ALU, control unit.
    - Example: Implementing pipelining for faster instruction execution.
  - **System Level**: Integration of CPU, memory, and I/O subsystems on a motherboard.
    - Example: Multi-core processors with shared caches.

Von Neumann Architecture

Von Neumann Architecture

- The most widely used computer architecture model.
- Describes a system where program instructions and data share the same memory space.
  - **Single Memory**: Both data and instructions are stored in the main memory.
  - **Sequential Execution**: Instructions are fetched one at a time and executed in order.
  - **Stored Program Concept**: Instructions are represented as binary data, just like regular data.
  - **Input/Output (I/O)**: Controlled via a central bus system.

Advantages	Disadvantages
• Simplicity in design. • Easy to reprogram (change instructions in memory).	**Von Neumann bottleneck**: CPU and memory share the same bus, limiting speed when fetching instructions and data simultaneously.

Stored Program Architecture

- The underlying principle behind the Von Neumann model: a program is stored in memory alongside data, and the CPU fetches instructions from memory sequentially.
  - **Fetch**: The next instruction is fetched from memory.
  - **Decode**: The control unit interprets the instruction.
  - **Execute**: The instruction is carried out (e.g., ALU operation, data transfer).

Buses

- A bus is a communication pathway that connects different components of the computer (CPU, memory, and I/O devices).
- Consists of a set of parallel wires used for transmitting data, addresses, and control signals.
- Types of Buses:
  - **Address Bus**: Carries memory addresses from CPU to memory/I/O.
    - Unidirectional (from CPU to memory).
    - Determines the maximum memory size.
    - Example: A 32-bit address bus allows addressing up to 2^{32} GB of memory.

- **Data Bus**: Transfers actual data between CPU, memory, and I/O.
  - Bidirectional (data can move both ways).
  - Width determines how many bits can be transferred at once.
  - Example: A 64-bit data bus can transfer 64 bits in a single cycle.
- **Control Bus**: Transmits control signals to coordinate operations.
  - Read/Write signals – Indicate data transfer direction.
  - Clock signals – Synchronize components.
  - Interrupt signals – Indicate urgent CPU attention to a spontaneous event
  - Example: a 8-bit control bus can have 8 control signals

Bus Cycle

- Synchronizes data transfers between various components in a computer's system, like the CPU, memory, and other peripherals, using the system bus.
- Typically measured in MHz.
- Calculated as `Bus Cycle Time = (1 / Bus Clock Rate)`

Bus Capacity (Bandwidth)

- The maximum rate at which data can be transferred over the bus, usually measured in bytes/sec or bits/sec.
- Calculated as `Bus Capacity = Data Transfer Unit * Bus Clock Rate`

Binary Data Organization

Units of Digital Information

TABLE 1. Units of digital information.

Unit	Description
Bit	**2 cells** (0 or 1). Abbreviated as lowercase b.
Crumb	A group of **2 bits**.
Nibble	A group of **4 bits**.
Byte	A group of **8 bits**. Abbreviated as uppercase B.
Word	A group of bits **defined by the computer architecture** (e.g., 16-bit, 32-bit, 64-bit). Words may also have extensions: • **Double Word**: twice the size of a word. • **Quad Word**: Four times the size of a word.

Least Significant vs. Most Significant

- **Least Significant:**
  - Refers to the rightmost position in a binary data organization.
  - Represents the **smallest weight or value**.
- **Most Significant:**
  - Refers to the leftmost position in a binary data organization.
  - Represents the **largest weight or value**.

```
Example 1: 0x42AB3156CAFE5687 where a word is defined as 16-bit.

(the prefix 0x denotes hexadecimal / base-16)

(remember that each digit in base-16 represents 4 bits)

LSNibble    -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = 7

MSNibble    -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = 4

LSByte      -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = 78

MSByte      -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = 12

LSWord      -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = 5678
```

```
MSWord      -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = 12AB

LSDWord     -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = CAFE5678

MSDWord     -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7 = 12AB3456

(turn the hexadecimal into binary to "see" the bit or crumb)

LSBit       -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7       (expand 7)
            -> 7 -> (base-2) -> 0 1 1 1        = 1   (last 1 bit)

MSBit       -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7       (expand 4)
            -> 4 -> (base-2) -> 0 1 0 0        = 0   (first 1 bit)

LSCrumb     -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7        (expand 7)
            -> 7 -> (base-2) -> 0 1 1 1        = 11  (last 2 bits)

MSCrumb     -> 4 2 A B 3 1 5 6 C A F E 5 6 8 7        (expand 4)
            -> 4 -> (base-2) -> 0 1 0 0        = 01  (first 2 bits)
```

Byte Ordering Systems (Endianness)

- Refers to the convention for **interpreting the byte order of multi-byte data** in memory.
- **Little Endian**:
  - Least significant byte (LSB) is stored at the lowest memory address.
  - Multi-byte data is **stored in ascending order** of significance.
  - The address of the multi-byte is the address of the LSB.

```
Example 1: 32-bit data 0x12345678 stored at address 0000

Address    Value
0003       12    |
0002       34    |
0001       56    |
0000       78    v

address 0000 contains 16-bit data = 5678
address 0000 contains 32-bit data = 12345678
```

- **Big Endian**:
  - Most significant byte (MSB) is stored at the lowest memory address.
  - Multi-byte data is **stored in descending order** of significance.
  - The address of the multi-byte is the address of the MSB.

```
Example 1: 32-bit data 0x12345678 stored at address 0000

Address    Value
0003       78    ^
0002       56    |
0001       34    |
0000       12    |

address 0000 contains 16-bit data = 1234
address 0000 contains 32-bit data = 12345678
```

FAQ: What happens if endianness is ignored?

? If endianness is not accounted for **during data exchange between systems** with different conventions, the interpretation of multi-byte data will be incorrect. For example, `0x12345678` in Big Endian may be misinterpreted as `0x78563412` in Little Endian.

2^x vs 10^y Ambiguity

2^x vs 10^y is a common area of confusion in computer science because of the different measuring systems that humans use compared to computers. As we use decimal-based measurements, it may seem difficult to adapt to the computer's binary computing systems.

Measurement Systems

- Binary systems such as **RAM memory and memory capacity** are measured in binary values while hardware such as **hard disk storage or network bandwidth** are measured using decimal values.
- Because of this, the binary measurement has its own set of prefixes that match our decimal measurement system.

Different Prefixes of Measurement Systems

Binary Measurement System	Bytes	Decimal Measurement System	Bytes
Kibibyte (KiB)	2^{10} = 1,024	**Kilobyte (KB)**	10^3 = 1000
Mebibyte (MiB)	2^{20} = 1,048,576	**Megabyte (MB)**	10^6 = 1,000,000
Gibibyte (GiB)	2^{30} = 1,073,741,824	**Gigabyte (GB)**	10^9 = 1,000,000,000
Tebibyte (TiB)	2^{40} = 1,099,511,627,776	**Terabyte (TB)**	10^{12} = 1,000,000,000,000

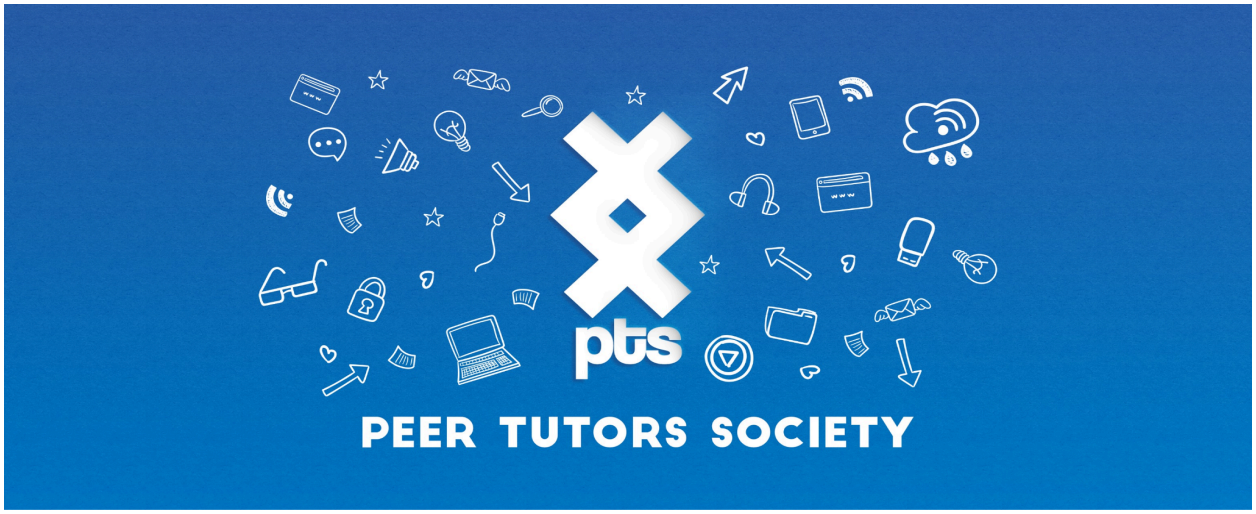
```
Example 1: If a PC build has 24GB of RAM, how many bytes of memory
does it have?

24GB of RAM -> 24 * 2^30
            -> 25,769,803,776 bytes or around 25.77GB of memory.

(note that 2^30 is 1GiB or 1,073,741,824 bytes)
```

```
Example 2: Your current WiFi has an upload speed of 20 Mbps, what
would be the speed in mebibytes?

20Mbps -> 20 * 10^6                          (megabits to bits)
       -> 20,000,000 bits per second
       -> 20,000,000 / 8                    (bits to bytes)
       -> 2,500,000 bytes per second
       -> 2,500,000 / 2^20                  (bytes to mebibytes)
       -> 2.3841..
       -> 2.38 mebibytes per second         (Rounded up final answer)
```



Verified and Checked by:

Anthony Baybayon - Vice President for Academics
anthony_baybayon@dlsu.edu.ph
Clive Jarel Ang - Associate Vice President for Academics
clive_jarel_c_ang@dlsu.edu.ph

Contact for Concerns:

Roemer Gabriel Caliboso - President
roemer_caliboso@dlsu.edu.ph
Anthony Baybayon - Vice President for Academics
anthony_baybayon@dlsu.edu.ph
Clive Jarel Ang - Associate Vice President for Academics
clive_jarel_c_ang@dlsu.edu.ph